

Parte 2 — Desenho, reamostragem e preparação

Tutorial completo do sciModelFlowR (versão 1.0.2)

Walter Esfrain Pereira

2026-09-19

Índice

1	Por que a reamostragem decide o resultado	3
2	Esquemas de reamostragem	4
3	O preço do vazamento, medido	7
4	Bootstrap: a unidade certa muda o intervalo	10
5	Preparação: imputação, centro e escala	13
6	Desbalanceamento: mexer só no treino	16
7	Seleção de atributos e representação	18
8	Exercícios da Parte 2	24

1 Por que a reamostragem decide o resultado

A pergunta científica é sempre “como este modelo se comporta em observações novas?”. A resposta depende inteiramente de **o que conta como observação nova**. Se talhões da mesma fazenda aparecem no treino e no teste, o modelo memoriza a fazenda e a estimativa fica otimista. Se o pré-processamento é ajustado no conjunto inteiro antes da divisão, a mediana do teste vaza para o treino. O `sciModelFlowR` trata a reamostragem como parte do **desenho**: `smf_design_spec()` declara grupos, blocos, tempo e coordenadas, e o pacote recusa esquemas incompatíveis em vez de avisar.

2 Esquemas de reamostragem

Cada conjunto simulado tem uma estrutura que pede um esquema próprio. A tabela mostra o que foi declarado e o que foi construído.

```
campo <- ler("field_yield")
fazendas <- ler("farm_groups")
clima <- ler("climate_series")
solo <- ler("soil_spatial")

h <- smf_holdout_split(campo, train_prop = 0.7, id_column = "plot_id", seed = 101L)
v <- smf_vfold_cv(campo, v = 5L, id_column = "plot_id", seed = 102L)
gv <- smf_group_vfold_cv(fazendas, groups = "farm", v = 6L, id_column = "plot_id", seed = 103L)
tc <- smf_time_cv(clima, time = "day", initial = 400L, assess = 100L, id_column = "day", seed = 104L)
sc <- smf_spatial_cv(solo, coords = c("x", "y"), v = 4L, id_column = "point_id", seed = 105L)

resumo_esq <- data.frame(
  esquema = c("holdout (70/30)", "v-fold (v = 5)", "group v-fold (fazenda)",
    "time series CV", "spatial CV"),
  particoes = c(1L, length(v@plits), length(gv@plits), length(tc@plits), length(sc@plits)),
  propriedade = c("uma divisão aleatória estratificável",
    "cada unidade no teste exatamente uma vez",
    "fazendas inteiras fora do treino",
    "passado treina, futuro testa",
    "blocos espaciais inteiros fora do treino")
)
tab_smf(resumo_esq, caption = "Esquemas de reamostragem construídos para cada estrutura de dados")
```

Tabela 2.1: Esquemas de reamostragem construídos para cada estrutura de dados.

esquema	particoes	propriedade
holdout (70/30)	1	uma divisão aleatória estratificável
v-fold (v = 5)	5	cada unidade no teste exatamente uma vez
group v-fold (fazenda)	6	fazendas inteiras fora do treino
time series CV	3	passado treina, futuro testa
spatial CV	4	blocos espaciais inteiros fora do treino

```
unidades <- 1:12
bloco <- function(nome, folds, papel_de) {
  do.call(rbind, lapply(folds, function(f) data.frame(
```

```

    scheme = nome, unit = unidades, fold = f, papel = papel_de(f)
  )))
}
esquema <- rbind(
  bloco("Holdout", 1L, function(f) ifelse(unidades <= 8, "treino", "teste")),
  bloco("V-fold", 1:3, function(f) ifelse(ceiling(unidades / 4) == f, "teste", "treino")),
  bloco("Group v-fold", 1:3, function(f) ifelse(ceiling(unidades / 4) == f, "teste", "treino")),
  bloco("Time series", 1:3, function(f) ifelse(unidades > 4 * f & unidades <= 4 * (f + 1),
                                                "teste", ifelse(unidades <= 4 * f, "treino", "treino")),
  bloco("Spatial blocks", 1:3, function(f) ifelse(ceiling(unidades / 4) == f, "teste", "treino"))
)
esquema$scheme <- factor(esquema$scheme, levels = c("Holdout", "V-fold", "Group v-fold",
                                                  "Time series", "Spatial blocks"))
ggplot(esquema, aes(unit, factor(fold), fill = papel)) +
  geom_tile(colour = "white", linewidth = .8) +
  facet_wrap(~scheme, ncol = 1) +
  scale_fill_manual(values = c(treino = cores_smf[["primaria"]],
                              teste = cores_smf[["destaque"]],
                              fora = "#e8e8e8")) +
  labs(x = "unidade experimental", y = "partição", fill = NULL,
       title = "Resampling schemes: what trains and what tests") +
  theme(legend.position = "top", panel.grid = element_blank())

```

Resampling schemes: what trains and what tests

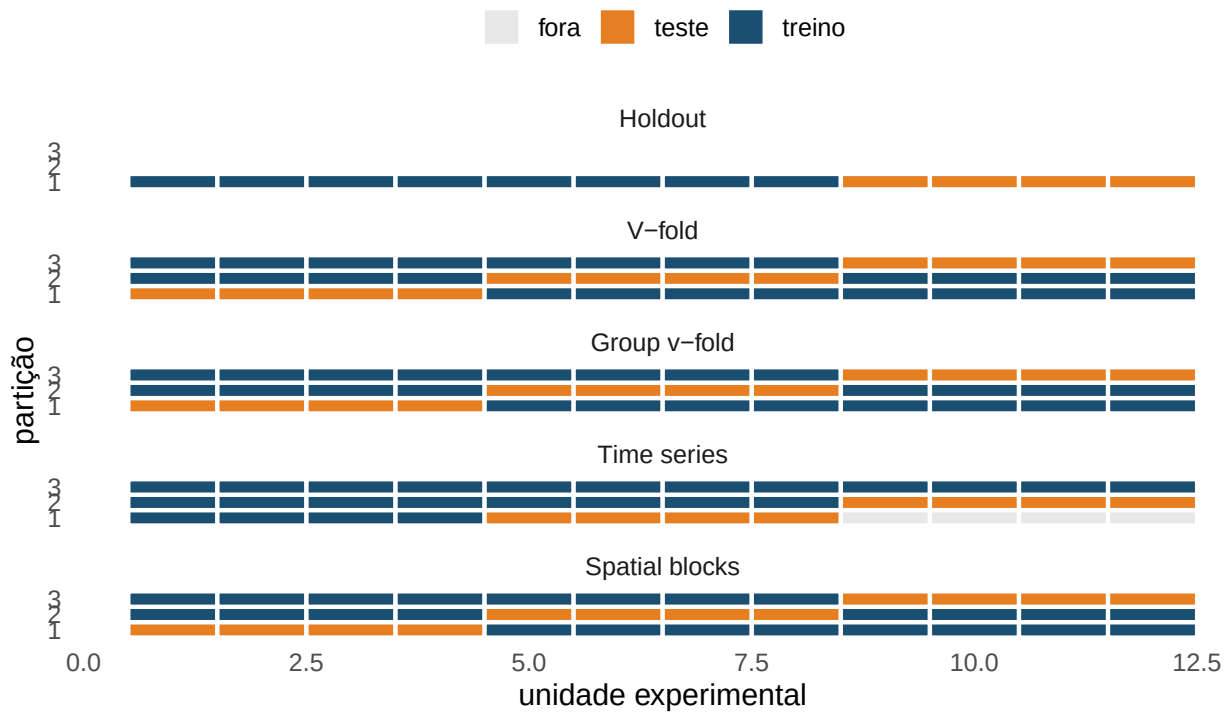


Figura 2.1: Como cada esquema parte os dados: treino em azul, teste em laranja. O esquema errado coloca vizinhos dos dois lados da fronteira.

i Interpretação

No v-fold comum, vizinhos de uma mesma fazenda (ou de um mesmo bloco espacial) aparecem dos dois lados da fronteira. Nos esquemas **group**, **time series** e **spatial**, a fronteira respeita a dependência: a fazenda inteira sai, o futuro não treina o passado, o bloco espacial inteiro é retirado. A escolha do esquema é uma decisão científica — e precisa ser declarada.

3 O preço do vazamento, medido

A melhor forma de aprender é ver o erro acontecer. O pacote **recusa** o v-fold comum quando há grupos declarados:

```
spec_iid <- smf_experiment_spec(
  task = smf_task_spec("regression", "yield"),
  data = smf_data_spec("yield", c("nitrogen", "farm"), id_column = "plot_id",
    categorical = "farm"),
  design = smf_design_spec(id_column = "plot_id", group_columns = "farm"),
  preprocessing = smf_preprocess_spec("median", center = TRUE, scale = TRUE),
  resampling = smf_resampling_spec("kfold", n_splits = 6L, seed = 11L),
  model = smf_model_spec("linear_regression", "stats"),
  metrics = list(smf_metric_spec("rmse")),
  reproducibility = smf_reproducibility_spec(11L)
)
recusa_vaz <- tryCatch(smf_resample_experiment(spec_iid, fazendas),
  error = function(e) conditionMessage(e))
cat(recusa_vaz, "\n")
```

Row-wise random resampling is incompatible with declared grouped/repeated units by default.

Para **quantificar** o otimismo, forçamos o mesmo esquema com `override = TRUE` e comparamos com o esquema correto. O modelo inclui a fazenda como preditor: com v-fold comum, o modelo aprende o efeito de cada fazenda nos outros talhões e acerta o teste; com `group`, a fazenda de teste é inédita.

```
spec_grp <- smf_experiment_spec(
  task = smf_task_spec("regression", "yield"),
  data = smf_data_spec("yield", c("nitrogen", "farm"), id_column = "plot_id",
    categorical = "farm"),
  design = smf_design_spec(id_column = "plot_id", group_columns = "farm"),
  preprocessing = smf_preprocess_spec("median", center = TRUE, scale = TRUE),
  resampling = smf_resampling_spec("group", n_splits = 6L, seed = 11L),
  model = smf_model_spec("linear_regression", "stats"),
  metrics = list(smf_metric_spec("rmse")),
  reproducibility = smf_reproducibility_spec(11L)
)
res_iid <- smf_resample_experiment(spec_iid, fazendas, override = TRUE)
res_grp <- smf_resample_experiment(spec_grp, fazendas)
rmse_iid <- res_iid@aggregate$mean[res_iid@aggregate$metric == "rmse"]
```

```
rmse_grp <- res_grp@aggregate$mean[res_grp@aggregate$metric == "rmse"]
```

```
dados_vaz <- data.frame(
  esquema = c("v-fold comum (forçado)", "group v-fold (correto)"),
  rmse = c(rmse_iid, rmse_grp)
)
ggplot(dados_vaz, aes(esquema, rmse, fill = esquema)) +
  geom_col(width = .55, show.legend = FALSE) +
  geom_hline(yintercept = verdade$farm_groups$residual_sd, linetype = "dashed",
    colour = cores_smf[["sucesso"]]) +
  geom_hline(yintercept = sqrt(verdade$farm_groups$residual_sd^2 +
    verdade$farm_groups$farm_sd^2),
    linetype = "dotted", colour = cores_smf[["alerta"]]) +
  geom_text(aes(label = fmt(rmse, 1)), vjust = -0.4) +
  scale_fill_manual(values = c(cores_smf[["alerta"]], cores_smf[["primaria"]])) +
  annotate("text", x = 2.45, y = verdade$farm_groups$residual_sd, hjust = 1, vjust = -0.4,
    label = "resíduo (120)", colour = cores_smf[["sucesso"]], size = 3.2) +
  annotate("text", x = 2.45, y = sqrt(verdade$farm_groups$residual_sd^2 +
    verdade$farm_groups$farm_sd^2), hjust = 1, vjust = -0.4,
    label = "resíduo + fazenda (323)", colour = cores_smf[["alerta"]], size = 3.2) +
  labs(x = NULL, y = "RMSE (kg/ha)",
    title = "Leakage costs: the same model, two resampling schemes") +
  ylim(0, 360)
```

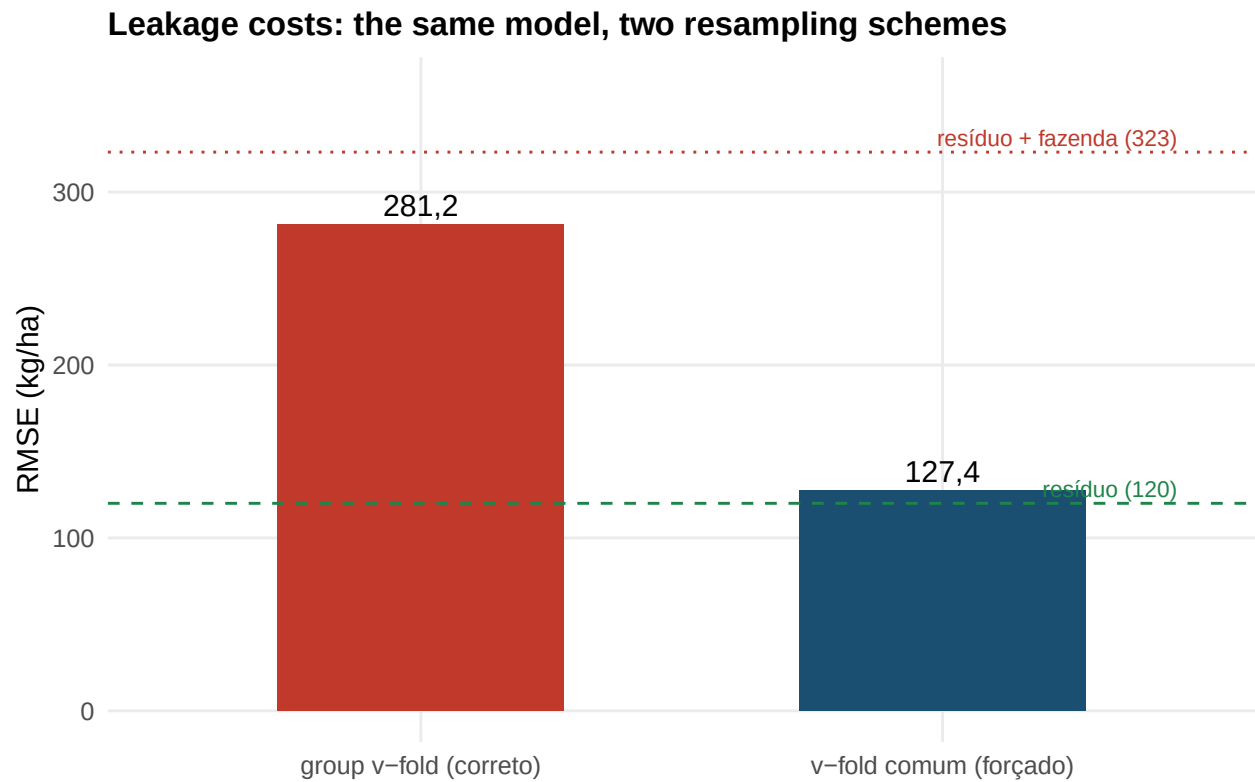


Figura 3.1: O mesmo modelo, dois esquemas: o v-fold comum (forçado) devolve RMSE muito menor que o group v-fold. As linhas marcam a verdade simulada.

! Interpretação

Com o esquema forçado, o RMSE cai para 127,4 kg/ha — abaixo até da dispersão residual verdadeira (120). É impossível: o número é otimista por construção, porque o modelo viu a mesma fazenda dos dois lados. Com **group**, o RMSE sobe para 281,2, compatível com a soma das duas fontes de variação (323). O pacote recusou o caminho errado; o **override** existe para **estudar** o problema, não para usá-lo em produção.

4 Bootstrap: a unidade certa muda o intervalo

O bootstrap responde “quanto esta estimativa varia se o experimento fosse repetido?”. A resposta só é honesta se a reamostragem respeitar a unidade que varia entre repetições. No ensaio com fazendas, a unidade é a **fazenda**, não o talhão.

```
b_case <- tryCatch(
  smf_bootstrap(fazendas, smf_bootstrap_spec("case", n_resamples = 99L,
                                              interval = "percentile", seed = 7L),
    statistic = function(z) c(media = mean(z$yield)),
    design = smf_design_spec(id_column = "plot_id",
                             group_columns = "farm")),
  error = function(e) conditionMessage(e))
b_bca_cl <- tryCatch(
  smf_bootstrap(fazendas, smf_bootstrap_spec("cluster", n_resamples = 99L,
                                              sampling_unit = "farm",
                                              interval = "bca", seed = 7L),
    statistic = function(z) c(media = mean(z$yield)),
    design = smf_design_spec(id_column = "plot_id",
                             group_columns = "farm")),
  error = function(e) conditionMessage(e))
cat("case (iid):", b_case, "\n\n")
```

case (iid): The requested bootstrap unit does not preserve dependence declared in DesignSpec.

```
cat("BCa com cluster:", b_bca_cl, "\n")
```

BCa com cluster: The generic row-delete BCa implementation is available only for case bootstrap aware schemes require a scheme-specific jackknife and are blocked rather than approximated.

```
b_cl <- smf_bootstrap(fazendas,
  smf_bootstrap_spec("cluster", n_resamples = 1999L, sampling_unit = "farm",
                      interval = "percentile", seed = 7L),
  statistic = function(z) c(media = mean(z$yield)),
  design = smf_design_spec(id_column = "plot_id", group_columns = "farm"))
ci_cl <- smf_bootstrap_interval(b_cl)
estab <- smf_bootstrap_stability(b_cl)
```

```
p1 <- ggplot(data.frame(media = b_cl@estimates[[1]]), aes(media)) +
  geom_histogram(fill = cores_smf[["primaria"]], colour = "white", bins = 35) +
  geom_vline(xintercept = ci_cl["media", ], linetype = "dashed",
    colour = cores_smf[["alerta"]]) +
  labs(x = "média do rendimento (kg/ha)", y = "reamostragens",
    title = "Cluster bootstrap distribution",
    subtitle = paste0("IC 95% percentil: [", fmt(ci_cl["media", 1], 0),
      "; ", fmt(ci_cl["media", 2], 0), "]"))
p2 <- ggplot(subset(estab, statistic == "media"), aes(B, width)) +
  geom_line(colour = cores_smf[["primaria"]]) +
  geom_point(size = 1.4) +
  labs(x = "reamostragens B", y = "largura do IC 95%",
    title = "Interval width stabilizes with B")
patchwork::wrap_plots(p1, p2)
```

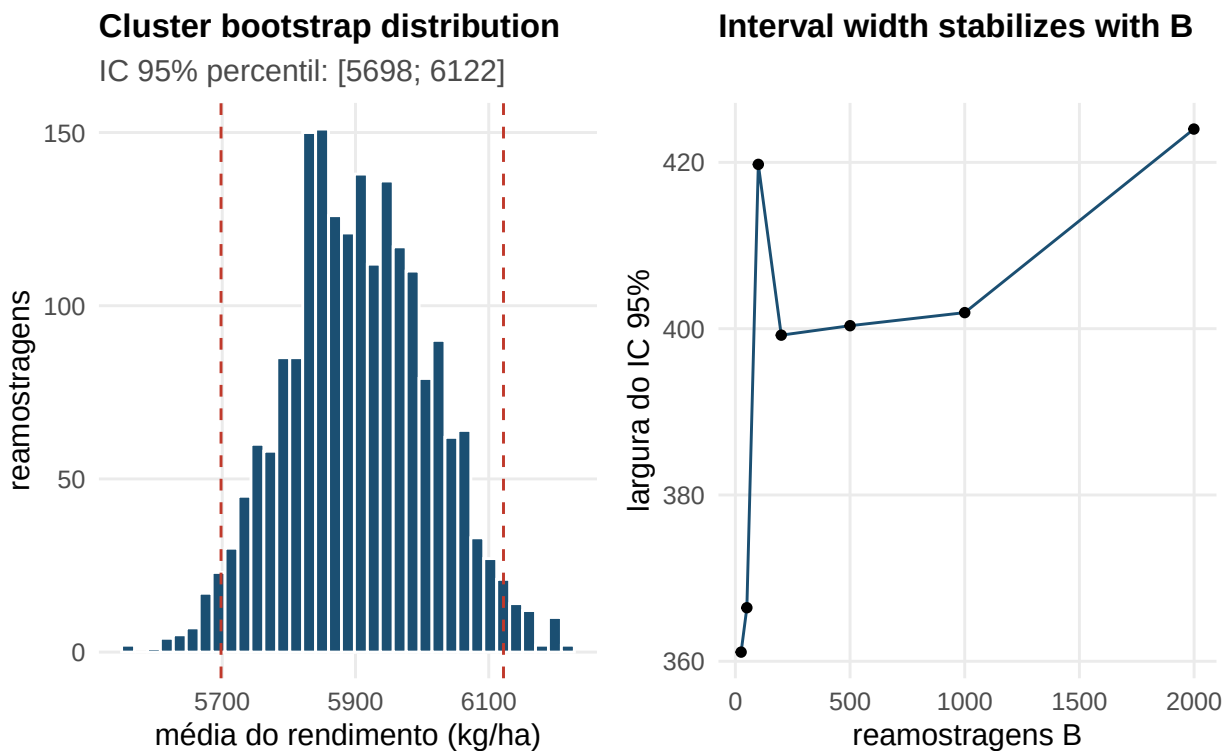


Figura 4.1: Distribuição bootstrap da média (unidade = fazenda) e largura do intervalo conforme B cresce: com 1999 reamostragens a largura já estabilizou.

⚠ Armadilha

O bootstrap **case** (talhão a talhão) foi **recusado** porque subestimaria a variabilidade: talhões da mesma fazenda não são repetições independentes. O BCa com cluster também foi recusado — a correção de viés exigiria um jackknife específico do esquema, e o pacote prefere bloquear a

aproximar. Os dois bloqueios são a mesma filosofia: **melhor um erro explícito do que um número plausível e errado.**

5 Preparação: imputação, centro e escala

A Parte 1 mostrou dois tipos de ausência: MCAR no nitrogênio e MAR (bloco B4) na matéria orgânica. Antes de comparar estratégias de imputação, porém, um passo obrigatório: **os outliers**. A tabela abaixo mostra o coeficiente da matéria orgânica (verdade plantada: 120) em quatro cenários.

```
sem_out <- campo[!campo$plot_id %in% verdade$field_yield$outlier_ids, ]
dspec <- smf_data_spec("yield", c("nitrogen", "water", "organic_matter"),
  id_column = "plot_id")
coef_om <- function(dados, imp) {
  pp <- smf_fit_preprocessor(dados, smf_preprocess_spec(imp), dspec, role = "training")
  tr <- smf_apply_preprocessor(pp, dados)
  tr[["yield"]] <- dados[["yield"]]
  fit <- lm(yield ~ nitrogen + water + organic_matter, data = tr)
  c(slope = unname(coef(fit)[["organic_matter"]]),
    n = sum(stats::complete.cases(tr)))
}
c_com <- coef_om(campo, "none")
c_none <- coef_om(sem_out, "none")
c_mean <- coef_om(sem_out, "mean")
c_med <- coef_om(sem_out, "median")
tab_imp <- data.frame(
  cenario = c("todos os dados, sem imputação",
    "sem outliers, sem imputação (casos completos)",
    "sem outliers, imputação pela média",
    "sem outliers, imputação pela mediana"),
  coef_mo = c(c_com["slope"], c_none["slope"], c_mean["slope"], c_med["slope"]),
  linhas_usadas = c(c_com["n"], c_none["n"], c_mean["n"], c_med["n"]))
)
tab_smf(tab_imp, caption = "Coeficiente da matéria orgânica sob diferentes decisões de tratamento")
```

Tabela 5.1: Coeficiente da matéria orgânica sob diferentes decisões de tratamento de discrepâncias (verdade plantada: 120).

cenario	coef_mo	linhas_usadas
todos os dados, sem imputação	285,808	153
sem outliers, sem imputação (casos completos)	115,286	149
sem outliers, imputação pela média	198,395	175
sem outliers, imputação pela mediana	206,427	175

```
ggplot(tab_imp, aes(stats::reorder(cenario, coef_mo), coef_mo)) +
  geom_col(fill = cores_smf[["primaria"]], width = .6) +
  geom_hline(yintercept = verdade$field_yield$beta[["organic_matter"]],
             linetype = "dashed", colour = cores_smf[["alerta"]]) +
  geom_text(aes(label = fmt(coef_mo, 1)), hjust = -0.15) +
  coord_flip() +
  labs(x = NULL, y = "coeficiente estimado (kg/ha por % de MO)",
       title = "Outliers first, imputation second") +
  ylim(0, max(tab_imp$coef_mo) * 1.15)
```

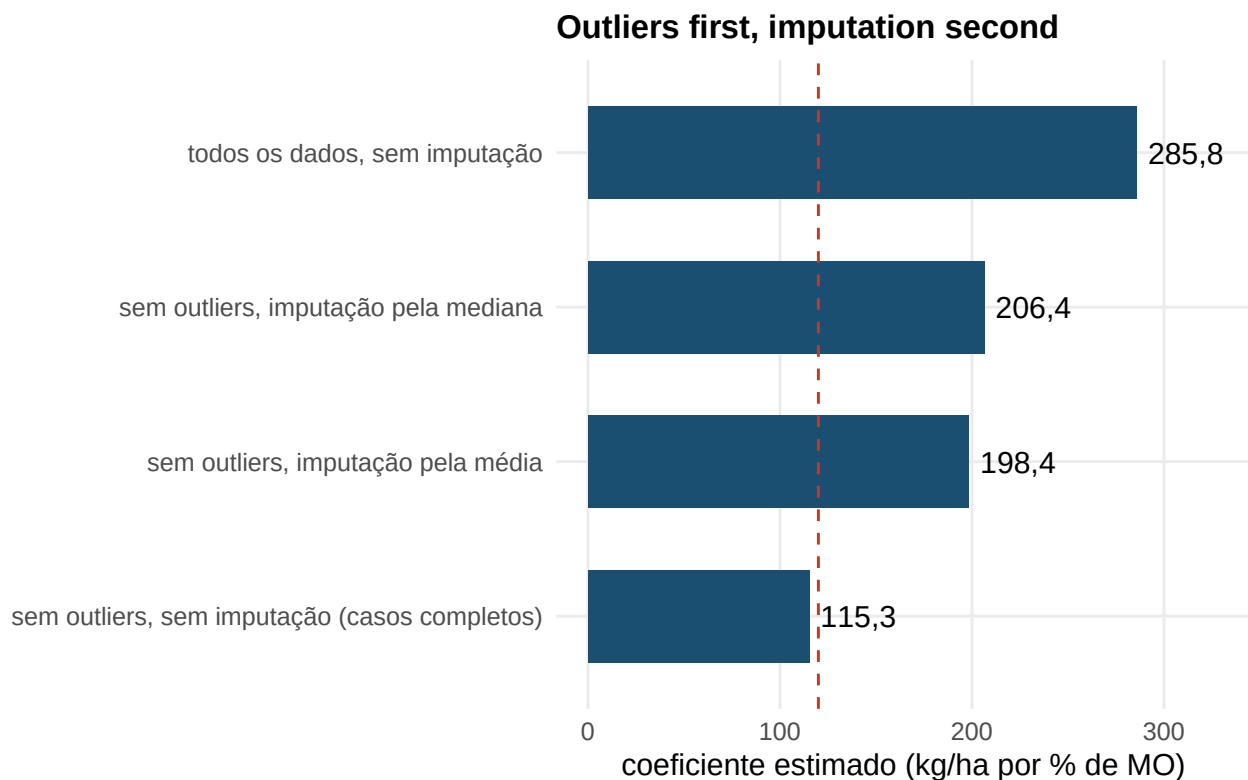


Figura 5.1: O coeficiente da matéria orgânica: os outliers dominam o erro; removidos, a escolha de imputação ainda desloca a estimativa.

! Interpretação

Três lições em uma tabela. (1) Com os cinco outliers dentro, o coeficiente quase triplica (286 contra a verdade 120) — discrepância numérica é o primeiro problema, não a ausência. (2) Removidos os outliers, casos completos recuperam 115,3, perto da verdade, mas usam apenas 149 das 180 parcelas. (3) A imputação pela média ou mediana usa todas as linhas, mas desloca o coeficiente para 198/206: imputar **não é neutro**. A decisão precisa ser declarada e testada em sensibilidade.

```

coef_escala <- function(center, scale) {
  pp <- smf_fit_preprocessor(sem_out, smf_preprocess_spec("median", center = center,
                                                         scale = scale),
                           dspec, role = "training")
  tr <- smf_apply_preprocessor(pp, sem_out)
  tr[["yield"]] <- sem_out[["yield"]]
  fit <- lm(yield ~ nitrogen + water + organic_matter, data = tr)
  round(coef(fit)[-1], 3)
}
tab_escala <- rbind(
  "sem centro, sem escala" = coef_escala(FALSE, FALSE),
  "com centro" = coef_escala(TRUE, FALSE),
  "com centro e escala" = coef_escala(TRUE, TRUE)
)
tab_smf(as.data.frame(tab_escala), caption = "Efeito de centrar e escalar nos coeficientes: c

```

Tabela 5.2: Efeito de centrar e escalar nos coeficientes: centrar não muda inclinações; escalar muda a unidade.

	nitrogen	water	organic_matter
sem centro, sem escala	24,004	2,744	206,427
com centro	24,004	2,744	206,427
com centro e escala	1044,576	302,382	91,741

💡 Boa prática

Centrar não altera inclinações (só desloca o intercepto). Escalar muda a unidade do coeficiente — e é indispensável para comparar a importância de preditores em escalas diferentes. O pacote ajusta o pré-processamento **dentro de cada fold**; nas Partes 3 e 4 isso será automático dentro de `smf_fit_experiment()` e `smf_resample_experiment()`.

6 Desbalanceamento: mexer só no treino

```
doenca <- ler("disease_binary")
doenca$disease <- factor(doenca$disease, levels = c(0, 1))
```

No conjunto de doença, a classe positiva é 13,8% e o ruído de rótulo limita o teto. As estratégias de balanceamento reamostram o **treino**; o teste permanece intocado.

```
tab_bal <- do.call(rbind, lapply(c("none", "downsample", "upsample", "smote"), function(m) {
  out <- tryCatch(smf_apply_imbalance(doenca, "disease", smf_imbalance_spec(m),
                                     context = "training", seed = 20260919),
                 error = function(e) conditionMessage(e))
  if (is.character(out)) {
    data.frame(metodo = m, classe_0 = NA_integer_, classe_1 = NA_integer_,
              observacao = out)
  } else {
    data.frame(metodo = m,
              classe_0 = sum(out@data$disease == "0"),
              classe_1 = sum(out@data$disease == "1"),
              observacao = "")
  }
}))
tab_smf(tab_bal, caption = "Contagens de classe antes e depois de cada estratégia de balanceamento")
```

Tabela 6.1: Contagens de classe antes e depois de cada estratégia de balanceamento (aplicada só ao treino).

metodo	classe_0	classe_1	observacao
none	345	55	
downsample	55	55	
upsample	345	345	
smote	NA	NA	Error in step_smote():

Caused by error in `prep()`: x All columns selected for the step should be double or integer. * 1 factor variable found: `plant_id` |

```
bal_longo <- data.frame(
  metodo = rep(tab_bal$metodo, 2),
  classe = rep(c("sadia (0)", "doente (1)"), each = nrow(tab_bal)),
  n = c(tab_bal$classe_0, tab_bal$classe_1)
)
ggplot(subset(bal_longo, !is.na(n)), aes(metodo, n, fill = classe)) +
  geom_col(width = .65) +
  scale_fill_manual(values = c("sadia (0)" = cores_smf[["claro"]],
                                "doente (1)" = cores_smf[["alerta"]])) +
  labs(x = NULL, y = "linhas no treino", fill = NULL,
       title = "Class balancing: training only") +
  theme(legend.position = "top")
```

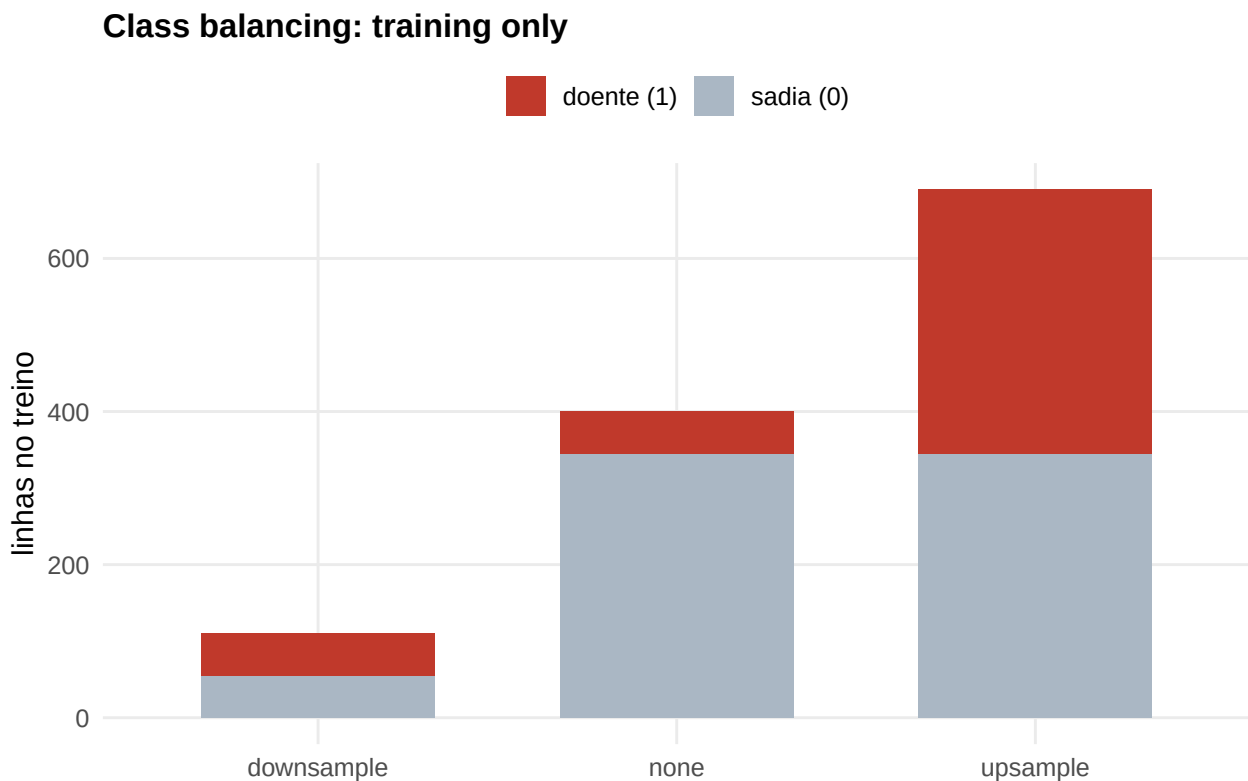


Figura 6.1: Balanceamento no treino: downsample iguala as classes reduzindo a maioria; upsample iguala replicando a minoria; o teste nunca é tocado.

⚠ Armadilha

Balancear **antes** de dividir contamina o teste: o mesmo registro pode ser duplicado no treino e cobrado no teste. O pacote exige `context = "training"` e recusa `context = "test"` (visto na Parte 1). SMOTE exige o desfecho como fator — o erro devolvido pelo pacote mostra exatamente isso.

7 Seleção de atributos e representação

Selecionar atributos com o conjunto inteiro é uma forma sutil de vazamento. O caminho correto é selecionar **dentro de cada fold**. O resultado traz as pontuações por fold e uma medida de estabilidade.

```
cv_campo <- smf_vfold_cv(sem_out, v = 5L, id_column = "plot_id", seed = 9L)
sel <- smf_feature_select_resamples(
  sem_out, "yield", smf_feature_spec("mutual_information", n_features = 3L),
  cv_campo, id_column = "plot_id"
)
tab_sel <- merge(sel@scores, sel@stability, by = "feature", all = TRUE)
tab_smf(tab_sel, caption = "Seleção por informação mútua (3 atributos por fold) repetida em 5
```

Tabela 7.1: Seleção por informação mútua (3 atributos por fold) repetida em 5 folds: frequência de seleção e estabilidade por atributo.

feature	selection_frequency	frequency
block	0,0	0,0
genotype	0,0	0,0
nitrogen	1,0	1,0
organic_matter	0,6	0,6
potassium	0,6	0,6
water	0,8	0,8

```
estab_sel <- sel@stability
names(estab_sel)[2] <- "frequencia"
ggplot(estab_sel, aes(stats::reorder(feature, frequencia), frequencia)) +
  geom_col(fill = cores_smf[["primaria"]], width = .6) +
  geom_text(aes(label = fmt(frequencia, 2)), hjust = -0.15, size = 3.2) +
  coord_flip() +
  labs(x = NULL, y = "frequência nos folds",
       title = "Feature selection stability across folds") +
  ylim(0, 1.12)
```

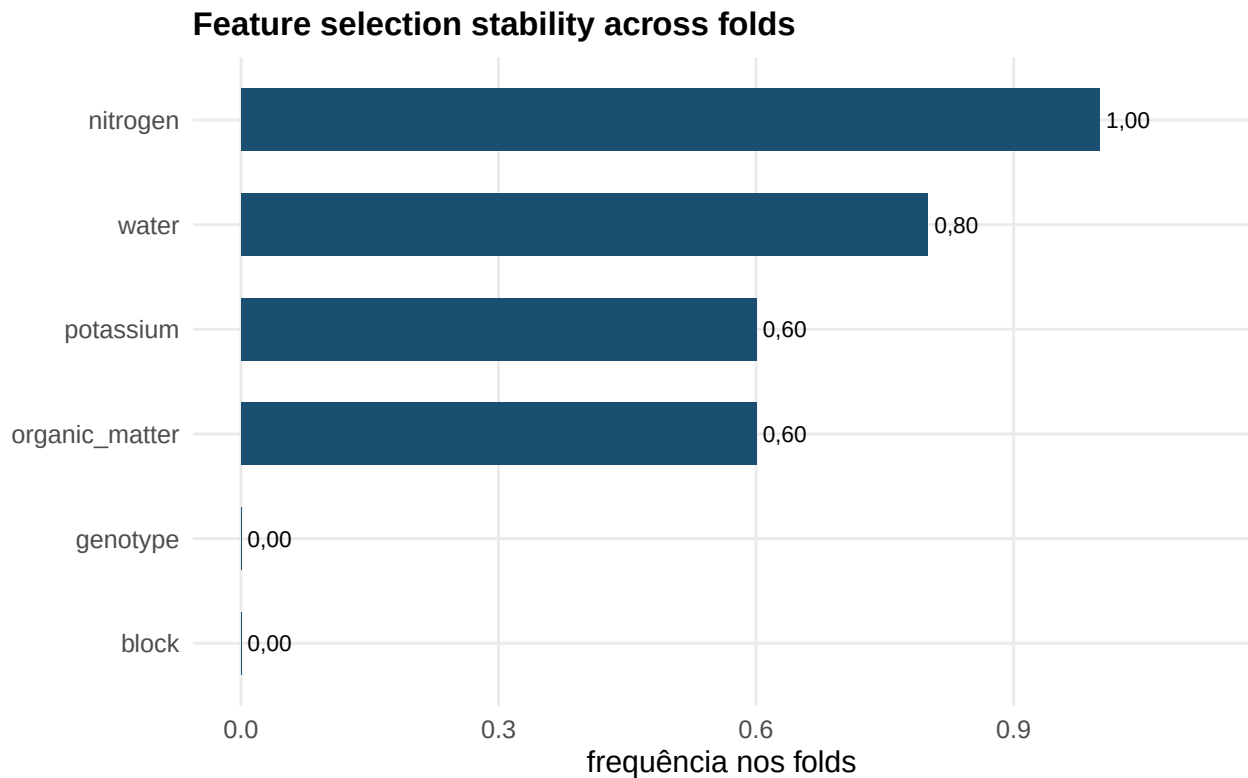


Figura 7.1: Estabilidade da seleção entre folds: atributos escolhidos em todos os folds são os candidatos confiáveis.

Em alta dimensão (200 comprimentos de onda para 120 amostras), a solução é **representação**: PCA ou PLS reduzem a dimensão antes do modelo, sempre ajustadas no treino. O experimento abaixo usa a representação PCA declarada na especificação.

```
espectro <- ler("spectral_pn")
ondas <- grep("^w", names(espectro), value = TRUE)
fazer_rep <- function(k) {
  spec <- smf_experiment_spec(
    task = smf_task_spec("regression", "protein"),
    data = smf_data_spec("protein", ondas, id_column = "sample_id"),
    design = smf_design_spec(id_column = "sample_id"),
    features = smf_feature_spec("pca", n_features = k, representation = "pca"),
    resampling = smf_resampling_spec("holdout", train_prop = 0.75, seed = 4L),
    model = smf_model_spec("linear_regression", "stats"),
    metrics = list(smf_metric_spec("rmse")),
    reproducibility = smf_reproducibility_spec(4L)
  )
  smf_fit_experiment(spec, espectro)
}
fit_rep <- fazer_rep(8L)
var_exp <- cumsum(fit_rep@fit@features@state$sdev^2) /
```

```

sum(fit_rep@fit@features@state$sdev^2)
tab_smf(data.frame(componente = seq_along(var_exp),
               var_acumulada = var_exp),
        caption = "Variância acumulada dos componentes principais ajustados no treino.")

```

Tabela 7.2: Variância acumulada dos componentes principais ajustados no treino.

componente	var_acumulada
1	0,468
2	0,756
3	0,992
4	0,993
5	0,993
6	0,993
7	0,994
8	0,994
9	0,994
10	0,994
11	0,995
12	0,995
13	0,995
14	0,995
15	0,995
16	0,996
17	0,996
18	0,996
19	0,996
20	0,996
21	0,996
22	0,997
23	0,997
24	0,997
25	0,997
26	0,997
27	0,997
28	0,997
29	0,998
30	0,998
31	0,998
32	0,998
33	0,998
34	0,998
35	0,998
36	0,998
37	0,998
38	0,998
39	0,999

componente	var_acumulada
40	0,999
41	0,999
42	0,999
43	0,999
44	0,999
45	0,999
46	0,999
47	0,999
48	0,999
49	0,999
50	0,999
51	0,999
52	0,999
53	0,999
54	0,999
55	0,999
56	0,999
57	1,000
58	1,000
59	1,000
60	1,000
61	1,000
62	1,000
63	1,000
64	1,000
65	1,000
66	1,000
67	1,000
68	1,000
69	1,000
70	1,000
71	1,000
72	1,000
73	1,000
74	1,000
75	1,000
76	1,000
77	1,000
78	1,000
79	1,000
80	1,000
81	1,000
82	1,000
83	1,000
84	1,000
85	1,000
86	1,000

componente	var_acumulada
87	1,000
88	1,000
89	1,000
90	1,000

```
rmse_comp <- do.call(rbind, lapply(c(2L, 4L, 6L, 8L, 12L, 20L), function(k) {
  f <- fazer_rep(k)
  data.frame(componentes = k,
              rmse = f@metrics$value[f@metrics$metric == "rmse"])
}))
tab_smf(rmse_comp, caption = "RMSE de validação por número de componentes principais (ruído v
```

Tabela 7.3: RMSE de validação por número de componentes principais (ruído verdadeiro: 0,25).

componentes	rmse
2	0,946
4	0,306
6	0,307
8	0,307
12	0,302
20	0,331

```
p_var <- ggplot(data.frame(componente = seq_along(var_exp), var = var_exp),
               aes(componente, var)) +
  geom_line(colour = cores_smf[["primaria"]]) +
  geom_point(size = 1.6) +
  labs(x = "componentes", y = "variância acumulada",
       title = "PCA variance (training fold)")
p_rmse <- ggplot(rmse_comp, aes(componentes, rmse)) +
  geom_line(colour = cores_smf[["destaque"]]) +
  geom_point(size = 1.6) +
  geom_hline(yintercept = 0.25, linetype = "dashed", colour = cores_smf[["alerta"]]) +
  labs(x = "componentes", y = "RMSE (validação)",
       title = "RMSE vs components")
patchwork::wrap_plots(p_var, p_rmse)
```

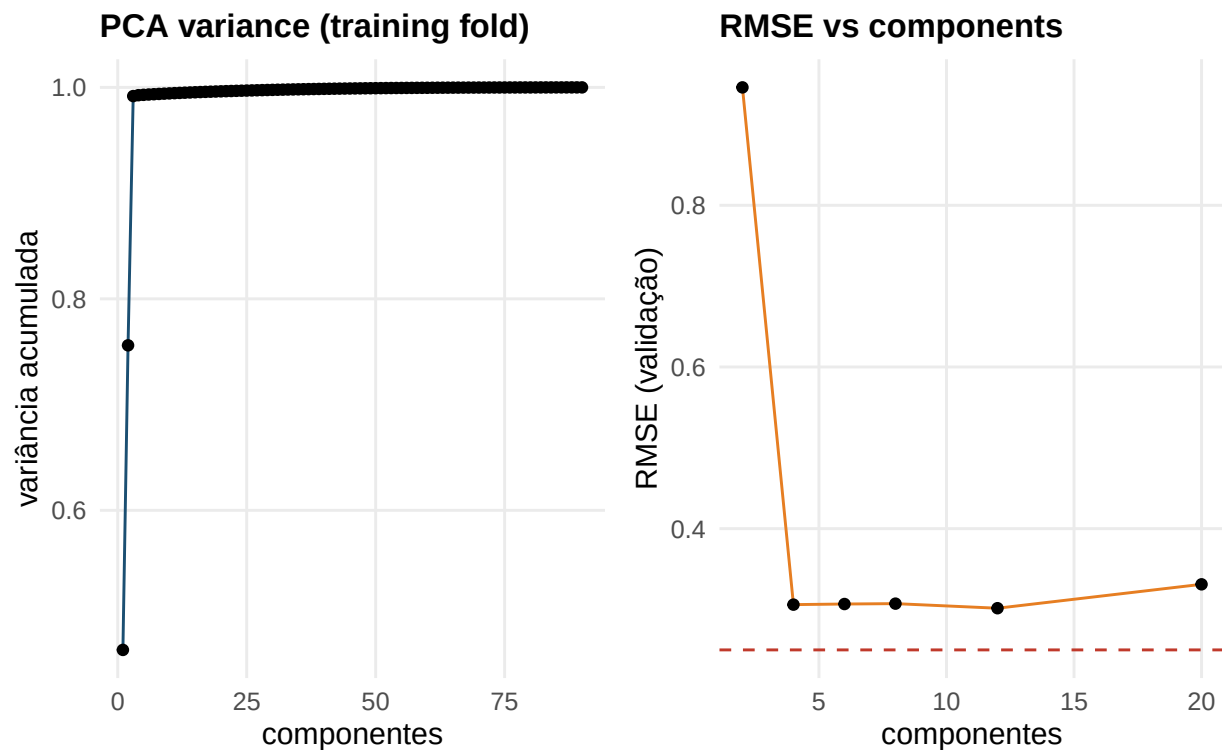


Figura 7.2: Representação por PCA: variância acumulada (esquerda) e RMSE de validação por número de componentes (direita).

i Interpretação

Com 120 amostras e 200 comprimentos de onda, o modelo linear direto é instável; a representação resolve. O RMSE de validação cai até 12 componentes (0,302) e depois se aproxima do ruído verdadeiro (0,25). Guarde este conjunto: na Parte 3 ele compara representações PCA e PLS, e na Parte 5 alimenta a explicabilidade.

8 Exercícios da Parte 2

Exercício 2.1 — Prove o vazamento

No `v-fold` comum sobre `farm_groups` (com `override = TRUE`), conte quantas fazendas aparecem nos dois lados de cada fold. Depois repita com o `group v-fold` e mostre que a contagem é zero. (Dica: `s@train_index`, `s@test_index` e `fazendas$farm`.)

Exercício 2.2 — Verifique a ordem temporal

No `smf_time_cv()` sobre `climate_series`, confirme que em **todos** os folds o dia de teste é posterior ao último dia de treino. Depois refaça com `smf_vfold_cv()` e mostre um caso em que a ordem é violada.

Exercício 2.3 — Quando o vazamento desaparece?

Repita a comparação `v-fold × group v-fold` **sem** a coluna `farm` nos preditores. O otimismo continua? Explique por quê, e o que isso ensina sobre quando o esquema errado é inofensivo (e por que ainda assim é arriscado).

Exercício 2.4 — MCAR contra MAR na imputação

Compare o coeficiente do nitrogênio (MCAR) e o da matéria orgânica (MAR) nas três estratégias de imputação. Qual dos dois coeficientes é mais sensível? Relacione com o padrão de ausência visto na Parte 1.

Soluções

2.1. Com `v-fold` comum forçado, cada fold tem talhões de fazendas que também estão no treino (sobreposição > 0 em todos os folds). No `group v-fold`, a interseção de fazendas treino teste é zero por construção.

2.2. No `smf_time_cv()`, `max(train) < min(test)` em todos os folds (expansão para o futuro). No `smf_vfold_cv()`, como a divisão é aleatória, algum fold terá dia de teste anterior a dias de treino — o modelo “aprende o futuro”.

2.3. Sem `farm` nos preditores, o modelo não consegue memorizar o efeito de fazenda: o otimismo praticamente desaparece. Mas continua arriscado: basta que exista **algum** atributo que carregue informação da fazenda (solo, clima local, prática de manejo) para o vazamento voltar. O desenho certo não depende de o modelo “não conseguir” explorar o vazamento.

2.4. O coeficiente da matéria orgânica (MAR, dependente do bloco) é mais sensível: as ausências concentram-se no B4, e imputar pela média global ignora essa estrutura. O nitrogênio (MCAR) sofre menos. O diagnóstico da Parte 1 (ausência por bloco) já previa isso.